

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Пермский национальный исследовательский политехнический
университет»

Электротехнический факультет

Кафедра: Информационные технологии и автоматизированные системы
(ИТАС)

Направление подготовки: 09.03.04 – «Программная инженерия»

ОТЧЕТ

Лабораторная работа №3.

Тема: «Перегрузка операций»

По дисциплине «Основы алгоритмизации и программирования»

Вариант 13

Выполнила: студентка группы РИС-25-26

Лаптева Елена Владимировна

Принял: доц. Полякова О.А.

Пермь, 2026

Постановка задачи:

Определить пользовательский класс.

Определить в классе следующие конструкторы:

- без параметров (конструктор по умолчанию)
- с параметрами
- копирования

Определить в классе деструктор.

Определить в классе компоненты-функции для просмотра и установки полей данных:

- селекторы (get-методы для просмотра)
- модификаторы (set-методы для установки)

Перегрузить операцию присваивания (=).

Перегрузить операции ввода и вывода объектов с помощью потоков (<< и >>).

Перегрузить операции, указанные в варианте.

Написать программу, в которой продемонстрировать:

- создание объектов
- работу всех перегруженных операций

Создать класс Pair (пара чисел)

Пара должна быть представлена двумя полями:

- типа int для первого числа
- типа double для второго числа

Первое число при выводе на экран должно быть отделено от второго числа двоеточием.

Реализовать:

- Операции сравнения: < и >
- Операцию декремента --, которая работает следующим образом:
 - если форма операции префиксная (--obj), то уменьшается первое число
 - если форма операции постфиксная (obj--), то уменьшается второе число

Анализ задачи:

1. Создаём класс `Pair` с двумя полями (`int` и `double`)
2. Реализуем все конструкторы, деструктор, `get/set` методы
3. Перегружаем операторы `=`, `<<`, `>>`, `<`, `>`, `--` (префиксный и постфиксный)
4. В `main()` показываем работу всего этого

Код:

```
#include <iostream>

using namespace std;

// КЛАСС PAIR (ПАРА ЧИСЕЛ)
class Pair {
private:
    int first;      // Первое число (целое)
    double second;  // Второе число (дробное)

public:
    // КОНСТРУКТОРЫ

    // 1. Конструктор по умолчанию (без параметров)
    Pair() : first(0), second(0.0) {
        // Инициализируем поля нулевыми значениями
    }

    // 2. Конструктор с параметрами
    Pair(int f, double s) : first(f), second(s) {
        // Инициализируем поля переданными значениями
    }

    // 3. Конструктор копирования
    Pair(const Pair& other) : first(other.first), second(other.second) {
        // Копируем значения полей из другого объекта
        // other – объект, который копируем
    }

    // ДЕКТРУКТОР
```

```

~Pair() {
    // В данном классе деструктор пустой
}

// СЕЛЕКТОРЫ (get-методы для просмотра полей)

int getFirst() const {
    // Возвращает первое число
    return first;
}

double getSecond() const {
    // Возвращает второе число
    return second;
}

// МОДИФИКАТОРЫ (set-методы для установки полей)

void setFirst(int f) {
    // Устанавливает первое число
    first = f;
}

void setSecond(double s) {
    // Устанавливает второе число
    second = s;
}

// ПЕРЕГРУЗКА ОПЕРАТОРОВ

// 1. Оператор присваивания (=)
Pair& operator=(const Pair& other) {
    if (this != &other) {
        first = other.first;    // Копируем первое число
        second = other.second;  // Копируем второе число
    }
    return *this; // Возвращаем текущий объект
}

```

```

// 2. Операторы сравнения (< и >)
bool operator<(const Pair& other) const {
    // Сравниваем пары чисел
    // Сначала сравниваем первые числа, если равны – вторые
    if (first != other.first)
        return first < other.first;
    return second < other.second;
}

bool operator>(const Pair& other) const {
    // Оператор > через оператор <
    return other < *this;
}

// 3. Оператор декремента (--)

// Префиксный декремент (--obj)
Pair& operator--() {
    // Уменьшаем первое число на 1
    first--;
    return *this; // Возвращаем измененный объект
}

// Постфиксный декремент (obj--)
Pair operator--(int) {
    // int параметр для отличия от префиксного
    Pair temp = *this; // Сохраняем текущее состояние
    second--;          // Уменьшаем второе число
    return temp;        // Возвращаем старое значение
}

// ДРУЗЬЯ (для доступа к private полям)

// Оператор вывода в поток (<<)
friend ostream& operator<<(ostream& out, const Pair& p);

// Оператор ввода из потока (>>)

```

```

    friend ostream& operator>>(ostream& in, Pair& p);
};

// РЕАЛИЗАЦИЯ ОПЕРАТОРОВ ВВОДА/ВЫВОДА

// Оператор вывода: формат "first:second"
ostream& operator<<(ostream& out, const Pair& p) {
    // Выводим первое число, затем двоеточие, затем второе
    out << p.first << ":" << p.second;
    return out; // Возвращаем поток для цепочки cout << a << b
}

// Оператор ввода
istream& operator>>(istream& in, Pair& p) {
    // Считываем два числа с клавиатуры
    cout << "Введите первое число (int): ";
    in >> p.first;
    cout << "Введите второе число (double): ";
    in >> p.second;
    return in; // Возвращаем поток
}

int main() {
    setlocale(LC_ALL, "ru");

    cout << "1. КОНСТРУКТОРЫ\n\n";

    // Конструктор по умолчанию
    cout << "Конструктор по умолчанию:\n";
    Pair p1;
    cout << "p1 = " << p1 << "\n\n";

    // Конструктор с параметрами
    cout << "Конструктор с параметрами (5, 3.14):\n";
    Pair p2(5, 3.14);
    cout << "p2 = " << p2 << "\n\n";

    // Конструктор копирования

```

```

cout << "Конструктор копирования (копируем p2):\n";
Pair p3 = p2;
cout << "p3 = " << p3 << "\n\n";

cout << "2. СЕЛЕКТОРЫ И МОДИФИКАТОРЫ\n\n";

// Используем get-методы
cout << "getFirst() для p2: " << p2.getFirst() << "\n";
cout << "getSecond() для p2: " << p2.getSecond() << "\n\n";

// Используем set-методы
cout << "Устанавливаем новые значения для p1 через set:\n";
p1.setFirst(10);
p1.setSecond(2.71);
cout << "p1 = " << p1 << "\n\n";

cout << "3. ОПЕРАТОР ПРИСВАИВАНИЯ\n\n";

Pair p4;
cout << "p4 до присваивания: " << p4 << "\n";
p4 = p1; // Используем перегруженный оператор =
cout << "p4 после p4 = p1: " << p4 << "\n\n";

cout << "4. ОПЕРАТОРЫ СРАВНЕНИЯ\n\n";

Pair p5(3, 1.5);
Pair p6(7, 2.8);
cout << "p5 = " << p5 << "\n";
cout << "p6 = " << p6 << "\n\n";

if (p5 < p6)
    cout << "p5 < p6 (истина)\n";
if (p6 > p5)
    cout << "p6 > p5 (истина)\n\n";

cout << "5. ОПЕРАТОР ДЕКРЕМЕНТА\n\n";

Pair p7(10, 5.5);

```

```

cout << "Исходный p7 = " << p7 << "\n";

// Префиксный декремент (уменьшает первое число)
cout << "\nПрефиксный --p7 (уменьшает первое число):\n";
cout << "До: " << p7 << "\n";
--p7;
cout << "После --p7: " << p7 << "\n";

// Постфиксный декремент (уменьшает второе число)
cout << "\nПостфиксный p7-- (уменьшает второе число):\n";
cout << "До: " << p7 << "\n";
p7--;
cout << "После p7--: " << p7 << "\n\n";

cout << "6. ВВОД С КЛАВИАТУРЫ\n\n";

Pair p8;
cout << "Введите значения для p8:\n";
cin >> p8; // Используем перегруженный оператор >>
cout << "Вы ввели: p8 = " << p8 << "\n\n";

cout << "7. ДЕМОНСТРАЦИЯ ВСЕХ ОБЪЕКТОВ\n\n";
cout << "p1 = " << p1 << "\n";
cout << "p2 = " << p2 << "\n";
cout << "p3 = " << p3 << "\n";
cout << "p4 = " << p4 << "\n";
cout << "p5 = " << p5 << "\n";
cout << "p6 = " << p6 << "\n";
cout << "p7 = " << p7 << "\n";
cout << "p8 = " << p8 << "\n";

return 0;
}

```


Результаты работы:

1. КОНСТРУКТОРЫ

Конструктор по умолчанию:

`p1 = 0:0`

Конструктор с параметрами (5, 3.14):

`p2 = 5:3.14`

Конструктор копирования (копируем p2):

`p3 = 5:3.14`

2. СЕЛЕКТОРЫ И МОДИФИКАТОРЫ

`getFirst()` для p2: 5

`getSecond()` для p2: 3.14

Устанавливаем новые значения для p1 через `set`:

`p1 = 10:2.71`

3. ОПЕРАТОР ПРИСВАИВАНИЯ

p4 до присваивания: 0:0

p4 после `p4 = p1`: 10:2.71

4. ОПЕРАТОРЫ СРАВНЕНИЯ

`p5 = 3:1.5`

`p6 = 7:2.8`

`p5 < p6` (истина)

`p6 > p5` (истина)

5. ОПЕРАТОР ДЕКРЕМЕНТА

Исходный `p7 = 10:5.5`

Префиксный `--p7` (уменьшает первое число):

До: `10:5.5`

После `--p7`: `9:5.5`

Постфиксный `p7--` (уменьшает второе число):

До: `9:5.5`

После `p7--`: `9:4.5`

6. ВВОД С КЛАВИАТУРЫ

Введите значения для `p8`:

Введите первое число (`int`): `34`

Введите второе число (`double`): `3.4`

Вы ввели: `p8 = 34:3.4`

7. ДЕМОНСТРАЦИЯ ВСЕХ ОБЪЕКТОВ

`p1 = 10:2.71`

`p2 = 5:3.14`

`p3 = 5:3.14`

`p4 = 10:2.71`

`p5 = 3:1.5`

`p6 = 7:2.8`

`p7 = 9:4.5`

`p8 = 34:3.4`

UML:

Pair
- first: int - second: double
+ Pair() + Pair(int, double) + Pair(const Pair&) + ~Pair() + getFirst(): int + getSecond(): double + setFirst(int): void + setSecond(double): void + operator=(const Pair&): Pair& + operator<(const Pair&): bool + operator>(const Pair&): bool + operator--(): Pair& + operator--(int): Pair + friend operator<<(ostream&, const Pair&): ostream& + friend operator>>(istream&, Pair&): istream&

Вопросы:

1. Для чего используются дружественные функции и классы? Если необходимо иметь доступ извне к скрытым полям класса, то есть расширить интерфейс класса, то можно использовать дружественные функции и дружественные классы.
2. Сформулировать правила описания и особенности дружественных функций. Дружественная функция объявляется внутри класса, к элементам которого ей нужен доступ, с ключевым словом `friend`. В качестве параметра ей должен передаваться объект или ссылка на объект класса, поскольку указатель `this` ей не передается. Дружественная функция может быть обычной функцией или методом другого ранее определенного класса. На нее не распространяется действие спецификаторов доступа, место размещения ее объявления в классе безразлично. Одна функция может быть дружественной сразу нескольким классам.
3. Каким образом можно перегрузить унарные операции? Унарную операцию можно перегрузить как компонентную функцию класса (нестатический метод без параметров) либо как внешнюю (глобальную) функцию.
4. Сколько операндов должна иметь унарная функция-операция, определяемая внутри класса? Унарная функция-операция, определяемая внутри класса, должна быть представлена с помощью нестатического метода без параметров (операндом является вызвавший её объект).
5. Сколько операндов должна иметь унарная функция-операция, определяемая вне класса? Если функция определяется вне класса, она должна иметь один параметр типа класса.
6. Сколько операндов должна иметь бинарная функция-операция, определяемая внутри класса? Бинарная функция-операция, определяемая внутри класса, должна быть представлена с помощью нестатического метода с одним параметром (вызвавший объект считается первым операндом, параметр – вторым).
7. Сколько операндов должна иметь бинарная функция-операция, определяемая вне класса? Если функция определяется вне класса, она должна иметь два параметра (оба операнда передаются явно).
8. Чем отличается перегрузка префиксных и постфиксных унарных операций? Операции постфиксного инкремента и декремента должны иметь первый параметр типа `int`. Он используется только для того, чтобы отличить их от префиксной формы.

9. Каким образом можно перегрузить операцию присваивания? Операция присваивания перегружается с помощью компонентной функции. Чтобы сохранить семантику присваивания, операция-функция должна возвращать ссылку на объект, для которого она вызвана, и принимать в качестве параметра единственный аргумент — ссылку на присваиваемый объект. В теле функции необходима проверка на самоприсваивание.

10. Что должна возвращать операция присваивания? Операция присваивания должна возвращать ссылку на объект, для которого она вызвана (обычно `return *this`).

11. Каким образом можно перегрузить операции ввода-вывода? Операции ввода-вывода `operator>>` и `operator<<` всегда реализуются как внешние дружественные функции, так как левым операндом этих операций являются потоки. Первый параметр — ссылка на поток, второй — ссылка на объект класса (для вывода — константная ссылка). Функция возвращает ссылку на поток.

12. В программе описан класс `class Student { ... Student& operator++(); ... }`; и объект `Student s`; выполняется `++s`. Как компилятор будет воспринимать вызов функции-операции? Компилятор воспримет это как вызов компонентной функции-операции `s.operator++()` (префиксная форма без параметров).

13. В программе описан класс `class Student { ... friend Student& operator++(Student&); ... }`; и объект `Student s`; выполняется `++s`. Как компилятор будет воспринимать вызов функции-операции? Компилятор воспримет это как вызов внешней (глобальной) дружественной функции `operator++(s)`.

14. Из-за приоритета операторов выражение `cout << a < b` будет воспринято как `(cout << a) < b`. Сначала выполнится `cout << a` (вывод объекта), затем результат этой операции попытается сравниться с `b` через оператор `<`. Это приведёт к ошибке компиляции, так как не существует оператора `<` между `ostream` и `Student`. Чтобы код работал правильно, нужно использовать скобки: `cout << (a < b)`; В этом случае компилятор воспримет `a < b` как вызов компонентной функции-операции: `a.operator<(b)`, получит результат типа `bool` (`true/false`), и затем выведет это значение (0 или 1).

15. Из-за приоритета операторов выражение `cout << a > b` будет воспринято как `(cout << a) > b`. Сначала выполнится `cout << a` (вывод объекта), затем результат этой операции попытается сравниться с `b` через оператор `>`. Это приведёт к ошибке компиляции, так как не существует оператора `>` между `ostream` и `Student`. Чтобы код работал правильно, нужно использовать скобки: `cout << (a > b)`; В этом случае компилятор воспримет `a > b` как вызов внешней

дружественной функции-операции: `operator>(a, b)`, получит результат типа `bool (true/false)`, и затем выведет это значение (0 или 1).